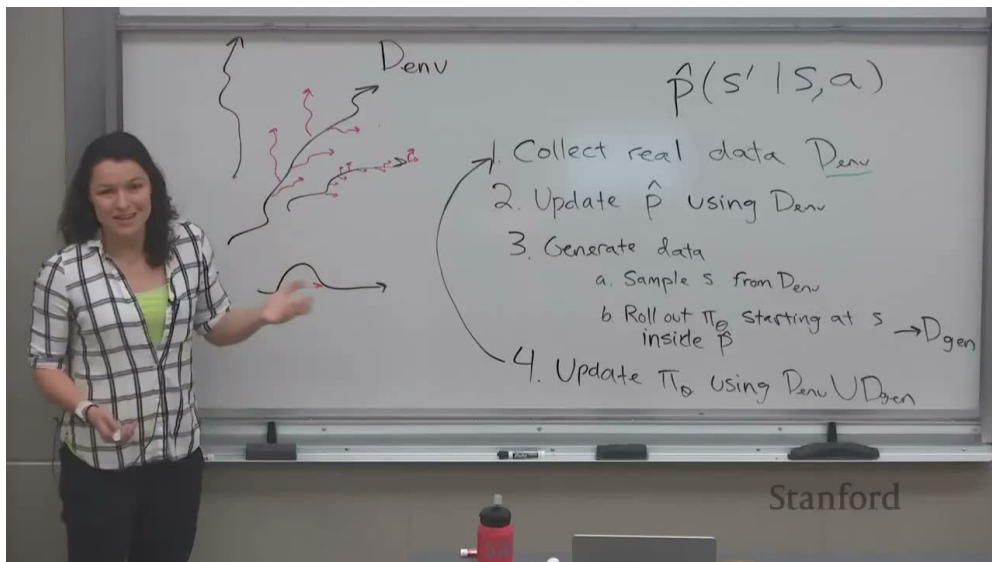


课程笔记

CS224R Lecture 12: Multi-Task and Goal-Conditioned RL

Codex

2026-04-13



视频作者/频道: Stanford CS224R

发布日期: 2025-05-09

视频时长: 约 01:10:29

视频链接: https://www.youtube.com/watch?v=qNdsI_4AQJw

目录

1	先补上上一讲未讲完的一半：如何用模型做 policy optimization	2
1.1	为什么 imagined rollouts 要短、而且要从真实状态出发	2
1.2	什么时候应该考虑 model-based RL	2
1.3	本章小结	3
2	为什么我们想学一个 generalist policy	3
2.1	multi-task RL 的动机不只是“一模多用”	3
2.2	generalist 往往比 specialist 更稳	4
2.3	本章小结	4
3	如何把“任务”写进 MDP	4
3.1	把不同任务当成不同 MDP	4
3.2	把 task identifier 放进状态里	5
3.3	本章小结	6
4	task conditioning 在实践中怎么做	6
4.1	从多任务模仿学习开始	6
4.2	任务标识可以是 one-hot，也可以是语言或示范	6
4.3	多任务算法只是在单任务算法外再接一个条件	7
4.4	本章小结	7
5	跨任务共享数据：从 hindsight relabeling 到 goal-conditioned RL	8
5.1	为什么可以把一条轨迹重解释成别的任务	8
5.2	goal-conditioned RL 是最自然的 relabeling 场景	8
5.3	但不是所有任务都能随便 relabel	9
5.4	本章小结	10
6	总结与延伸	10

1 先补上上一讲未讲完的一半：如何用模型做 policy optimization

这讲虽然题目是 multi-task 与 goal-conditioned RL，但开头并不是直接跳入多任务，而是先承接上一讲，补完 model-based policy optimization 的另一种用法。原因很合理：如果 planning 只适合短 horizon、测试时又太贵，那么 learned model 更现实的作用往往不是直接在线规划，而是帮助训练策略。

1.1 为什么 imagined rollouts 要短、而且要从真实状态出发

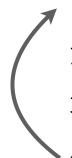
讲者首先回顾了 planning with learned models 的优缺点：优点是简单，可以直接换目标和奖励；缺点是测试时算力重，而且对长 horizon 不现实，因为既贵又会因模型误差累积而失真。于是自然的问题是：**能不能把模型的价值转移到训练阶段，而不是测试阶段？**

答案是肯定的。model-based policy optimization 的关键直觉，是用 learned model 生成合成数据，补充真实环境里拿到的轨迹。但讲者反复强调，不能从初始状态一口气生成很长 imagined trajectory；更好的做法，是从 replay buffer 里的很多真实状态出发，只生成**短的** synthetic rollouts。

Model-based policy optimization

Key idea: augment data with model-simulated roll-outs.

Full algorithm

- 
1. Collect data using current policy π_θ , add to D_{env}
 2. Update model $p_\psi(s'|s, a)$ using D_{env}
 3. Collect short synthetic roll-outs using π_θ in model p_ψ from states in D_{env} ; add to D_{model}
 4. Update policy π_θ (and critic Q_ϕ) using $D_{env} \cup D_{model}$

Notes: - compatible with variety of model-free RL methods (step 4)
- could additionally use D_{env} in policy update

7

图 1: model-based policy optimization 的完整流程：环境数据训练模型，模型从真实状态附近生成短 imagined rollouts，再反过来更新策略。

这种做法的好处有两个。第一，它让合成轨迹始终锚定在真实数据邻域，不容易完全漂走。第二，它把模型误差控制在短 horizon 内，更适合作为局部覆盖增强，而不是全盘替代环境。

1.2 什么时候应该考虑 model-based RL

课程没有把 model-based RL 当作默认更高级的方法，而是给出一套非常明确的取舍标准：如果模型容易学，它能极大提升样本效率，并且还能在没有 reward 标签时用自监督方式训练；如果模型难学，它就会变成新的误差源、新的超参数源和新的计算负担。

When to use model-based RL?

Big upsides and big downsides

- + Immensely useful, far more data efficient if model is easy to learn
- + Model can be trained on data without reward labels (fully self-supervised)
- + Model is somewhat task-agnostic (can sometimes be transferred across rewards)
- Models don't optimize for task performance
- Sometimes harder to learn than a policy
- Another thing to train, more hyperparameters, more compute intensive

Whether to use a model depends on how hard it is to learn!

8

图 2: 是否使用 model-based RL, 核心取决于模型是否足够好学, 而不是它在概念上是否“更聪明”。

这一点和后半节的多任务主题其实是相通的: 无论是 learned dynamics 还是 shared multi-task structure, 真正有用的前提都是存在可复用、可泛化的结构。如果底层结构本身不存在, 共享和迁移就无从谈起。

1.3 本章小结

Lecture 12 的开头先把上一讲的故事补完整了。规划不是模型的唯一用途; 在很多现实系统中, 模型更像一个训练期的数据放大器。只要 imagined rollouts 足够短、足够贴近真实分布, 它们就能为策略学习提供额外的局部探索信号。

2 为什么我们想学一个 generalist policy

2.1 multi-task RL 的动机不只是“一模多用”

讲者随后把话题转到 multi-task RL。表面上看, 多任务的目标很直观: 不想只学一个会做单一事情的策略, 而是想学一个 generalist policy, 能处理多个目标、多个任务、多个用户或多个环境。例子横跨语言助手、腿足机器人、移动机械臂、推荐系统和游戏代理。

What is multi-task RL?

Can we train generalist policies to do many “tasks”, not just one?

Examples: Generalist LLM assistant: travel booking, grocery shopping, ...
Legged robot: walk, run, dance, crouch, ...
Mobile manipulator: hang up a towel, unload a dishwasher, wipe a spill, ...
Music recommender system: personalization to many distinct users
Game player: play Flappy bird, Pokemon, Pinball, ...

May have different reward functions, different dynamics, different action spaces, ...!

Key idea will be to condition on the task.

11

图 3: multi-task RL 关注的是 generalist policy: 一个策略需要在任务标识变化时展现出不同能力。

但课程真正强调的不是“方便”，而是**数据效率**。如果多个任务共享结构，例如语言助手共享语法与网页操作模式，腿足机器人共享平衡能力，不同用户推荐共享偏好簇结构，那么多个任务联合训练，就可能把单任务本来需要反复独立学习的知识 amortize 掉。

2.2 generalist 往往比 specialist 更稳

讲者给出的第二个动机更值得注意：经验上，generalist systems 往往不只是覆盖面更广，还常常比 specialist 更可靠、更高性能。这并非逻辑必然，但在近年 AI 系统里屡次出现。课程用它来说明：多任务并不只是工程上的合并，同样可能是性能提升的来源。

这节课的关键转折

从这里开始，课程把“如何学一个任务”变成“如何在共享结构中同时学多个任务”。这意味着状态、动作、奖励和任务标识之间的关系必须重新形式化，否则多任务只会停留在口号层面。

2.3 本章小结

multi-task RL 的根本目标不是省模型数量，而是利用任务之间的共享结构提升样本效率和泛化。只有把这种共享写进问题形式里，后续的 task conditioning、relabeling 和 goal-conditioned RL 才有意义。

3 如何把“任务”写进 MDP

3.1 把不同任务当成不同 MDP

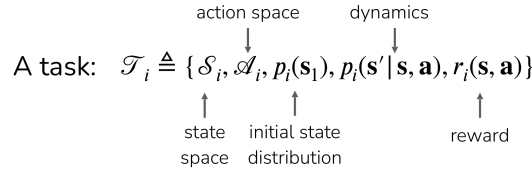
课程给出的第一种形式化方式，是把每个任务 i 看成一个独立的 MDP：

$$\mathcal{M}_i = (\mathcal{S}_i, \mathcal{A}_i, p_i(s_1), p_i(s'|s, a), r_i(s, a)).$$

这里不同任务可以共享某些组成部分，也可以在状态空间、动作空间、动力学、初始状态分布、奖励函数上彼此不同。讲者特别提醒，这种定义比日常语言里的“任务”更一般，因为它允许我们把任何 MDP 差异都视为任务差异。

How do we formalize what different “tasks” are?

Different tasks can just be different MDPs!



Note: This allows for much more variety than the semantic meaning of “task”!

13

图 4: 把任务形式化为一组不同的 MDP：状态、动作、动力学、初始状态分布与奖励函数都可能因任务而变化。

这个定义的价值，在于它直接告诉我们多任务共享可能发生在哪一层。比如不同 locomotion skill 往往共享动力学但奖励不同；不同 garment manipulation 任务则可能连动力学和初始状态分布都变；不同用户推荐的任务，奖励偏好会变，但系统交互接口可能保持相同。

3.2 把 task identifier 放进状态里

如果多个任务至少共享某个统一的表示空间，课程就给出第二种更便于算法实现的写法：把任务标识 z_i 直接并入状态，得到聚合后的 MDP。此时状态写作

$$s = (\bar{s}, z_i),$$

相应的奖励与动力学也写成依赖任务标识的形式：

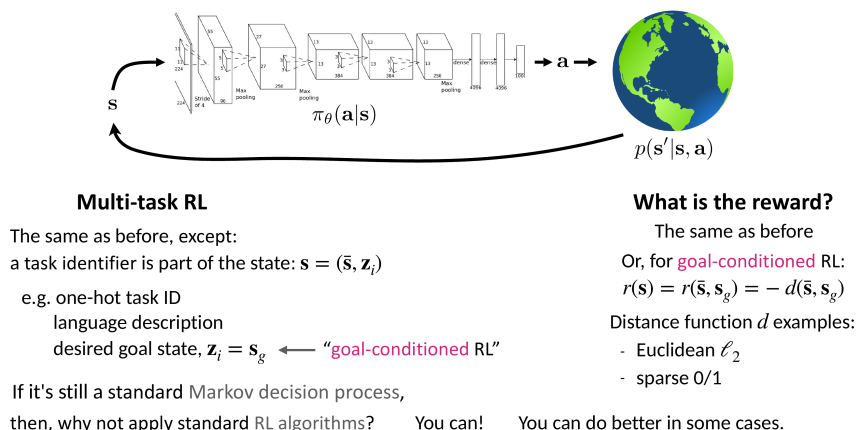
$$r((\bar{s}, z_i), a) = r_i(\bar{s}, a), \quad p((\bar{s}', z_i) | (\bar{s}, z_i), a) = p_i(\bar{s}' | \bar{s}, a).$$

对于 goal-conditioned RL，则常把任务标识具体化为目标状态 g 。这时一个常见奖励形式是

$$r(\bar{s}, g) = -d(\bar{s}, g),$$

也就是用当前状态与目标之间的距离来定义回报。距离既可以是 Euclidean ℓ_2 ，也可以是稀疏 0/1 成功信号。

The goal of multi-task reinforcement learning



17

图 5: 把 task identifier 并入状态后, 多任务问题就可以转写为一个扩展状态空间上的标准 RL 问题。

讲者明确指出: 一旦任务标识成为状态的一部分, **标准 RL 算法其实就可以直接用**。这为后面所有 multi-task imitation、multi-task RL、goal-conditioned RL 提供了统一入口。

3.3 本章小结

这一部分完成了多任务问题的数学重写。课程先把不同任务视为不同 MDP, 再说明只要能把任务标识嵌入状态, 就能把多任务问题重新变成一个“标准”的 RL 问题。后续的关键便转向: 这个 task identifier 在实践中怎么表示、怎么使用, 以及数据能否在任务之间共享。

4 task conditioning 在实践中怎么做

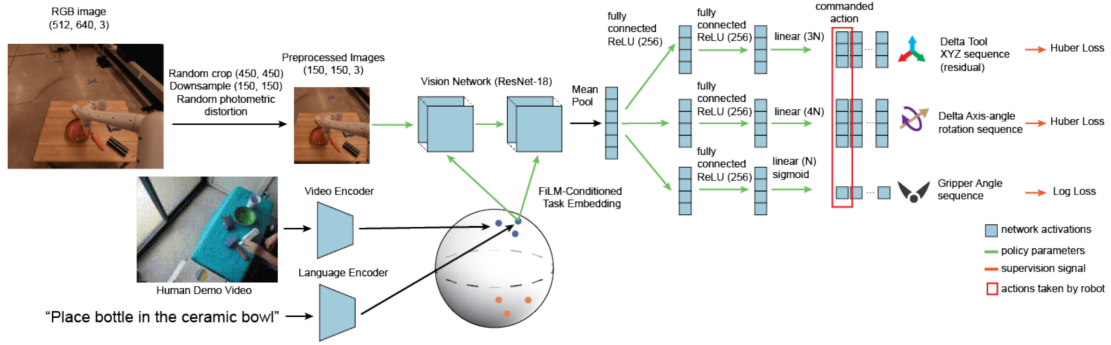
4.1 从多任务模仿学习开始

课程先从 multi-task imitation learning 讲起, 因为它最容易看清 task conditioning 的本质。对于单任务 BC, 策略只看状态; 到了多任务版本, 策略要看状态和任务标识。一个简单但很重要的训练细节, 是在每个 minibatch 里做分任务的 stratified sampling, 降低梯度方差, 避免数据量大的任务淹没数据量小的任务。

4.2 任务标识可以是 one-hot, 也可以是语言或示范

讲者没有把 task identifier 限制成 one-hot ID。相反, 课程明确展示了多种可能的 task conditioning 形式: one-hot 任务编号、语言描述、示范视频、目标状态, 甚至在更现代的系统里, 任务标识可以作为 prompt 输入给大模型策略。

How to condition on the task in practice?



$$\min \sum_{\text{task } i} \sum_{\substack{(s,a) \sim \mathcal{D}_e^i \\ w_h \sim \mathcal{D}_h^i \cup \mathcal{D}_e^i}} -\log \pi(a|s, z^i), \text{ where } \underbrace{z_h^i \sim q(\cdot|w_h)}_{\text{video encoder}}, \underbrace{z_\ell^i \sim q(\cdot|w_\ell^i)}_{\text{language encoder}}$$

Jang et al. BC-Z: Zero-Shot Task Generalization with Robotic Imitation Learning CoRL 2021.

19

图 6: task conditioning 的实践形态并不单一：视觉输入、示范视频、语言描述都可以共同构成任务嵌入。

这部分非常贴近当代 VLA 与 language-conditioned robotics 的做法。任务不再是一个抽象编号，而是一个可学习的上下文变量。讲者还提到，如果任务本质上是“不同人的偏好”，那么更好的表示甚至可能是几条该用户喜欢或不喜欢的例子，这就有点接近 in-context learning 的味道了。

4.3 多任务算法只是在单任务算法外再接一个条件

从算法视角看，multi-task RL 并没有完全改写 RL 的基本对象，而是在 policy 和 Q-function 上显式加入任务条件：

$$\pi_\theta(a | \bar{s}) \longrightarrow \pi_\theta(a | \bar{s}, z_i),$$

$$Q_\phi(\bar{s}, a) \longrightarrow Q_\phi(\bar{s}, a, z_i).$$

这说明一个很重要的事实：多任务 RL 的难点不在公式是否复杂，而在任务之间哪些数据真能共享，哪些共享会制造伪相关。

4.4 本章小结

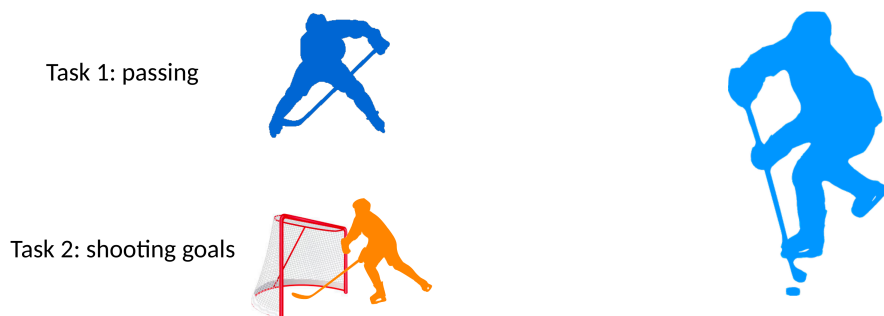
task conditioning 的本质，是把“当前应该做哪件事”变成策略输入的一部分。形式上这看似只是多加一个条件变量，实际上它决定了模型能否正确对齐不同任务语义，也决定了后续跨任务复用数据是否可能。

5 跨任务共享数据：从 hindsight relabeling 到 goal-conditioned RL

5.1 为什么可以把一条轨迹重解释成别的任务

课程接着讨论一个极有启发性的想法：如果一条轨迹本来是为了任务 z_1 采集的，但它在执行过程中顺便完成了任务 z_2 ，那这条数据能不能也拿来训练任务 z_2 ？这就是 hindsight relabeling 的基本直觉。

An example



What if you accidentally perform a good pass when trying to shoot a goal?

Store experience as normal. *and* Relabel experience with task 2 ID & reward and store.
"hindsight relabeling" "hindsight experience replay" (HER)

图 7: hindsight relabeling 的直觉：一条轨迹若意外完成了另一个任务，也可以被重新标注后再次利用。

在一般 multi-task RL 中，重标注可以写成：先收集

$$\mathcal{D}_k = \{(s_{1:T}, a_{1:T}, z_i, r_{1:T})\},$$

然后把它改写为另一个任务 z_j 下的版本

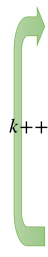
$$\mathcal{D}'_k = \{(s_{1:T}, a_{1:T}, z_j, r'_{1:T})\}, \quad r'_t = r_j(s_t).$$

只要新的任务奖励可以在事后计算，这种 relabeling 就让同一条经验服务于多个任务。

5.2 goal-conditioned RL 是最自然的 relabeling 场景

goal-conditioned RL 是 hindsight relabeling 最经典也最自然的应用。因为在目标到达任务里，轨迹里出现过的某个后续状态，本身就可以被重新解释为“原来这条轨迹的目标就是到这里”。

Goal-conditioned RL with hindsight relabeling

- 
1. Collect data $\mathcal{D}_k = \{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}, \mathbf{s}_g, r_{1:T})\}$ using some policy
 2. Store data in replay buffer $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_k$
 3. Perform **hindsight relabeling**:
 - a. Relabel experience in $\mathcal{D}_k$ using last state as goal:
 $\mathcal{D}'_k = \{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}, \mathbf{s}_T, r'_{1:T})\}$ where $r'_t = -d(\mathbf{s}_t, \mathbf{s}_T)$
 - b. Store relabeled data in replay buffer $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}'_k$
 4. Update policy using replay buffer $\mathcal{D}$
- <— Other relabeling strategies?
use **any state** from the trajectory

Result: exploration challenges alleviated

Kaelbling. Learning to Achieve Goals. IJCAI'93
Andrychowicz et al. Hindsight Experience Replay. NeurIPS'17

26

You will implement hindsight
relabeling in homework 4!

图 8: goal-conditioned RL with hindsight relabeling: 把轨迹终点或中间状态改写成目标, 就能把失败探索变成成功样本。

课上给出的算法流程非常清楚:

1. 收集一条以目标 g 为条件的轨迹;
2. 存进 replay buffer;
3. 事后把终点状态 s_T 或轨迹中的其他状态改写成新的目标;
4. 用新目标重新计算奖励, 再把重标注后的轨迹也放回 buffer;
5. 用 off-policy RL 从混合 buffer 中更新策略。

它最核心的收益, 在于把原本稀疏、经常失败的探索, 转化为事后看来成功的训练样本。这就是 HER, Hindsight Experience Replay 的经典思想。

5.3 但不是所有任务都能随便 relabel

讲者也明确指出, relabeling 并非普适魔法。它成立需要若干前提:

- 不同任务至少要共享兼容的状态空间与动作空间;
- 奖励函数形式必须已知且可事后评估;
- 动力学必须与新的任务解释相容;
- 通常需要 off-policy 算法, 因为 replay buffer 里会混入许多并非按当前任务采样的经验。

什么时候 relabeling 最有价值

当任务本身稀疏、探索难, 而且“失败轨迹”中蕴含大量潜在成功子目标时, relabeling 的价值最大。goal-conditioned robotic manipulation 正是这种结构最明显的场景。

5.4 本章小结

跨任务共享数据并不是简单地把所有轨迹混在一起，而是要利用任务结构进行有约束的重解释。hindsight relabeling 的妙处，在于它利用了任务之间的共同动力学，把“没完成原目标”的轨迹，转化成“完成了另一个可行目标”的样本，从而显著缓解探索困难。

6 总结与延伸

Lecture 12 实际上做了两件彼此呼应的事。前半段说明 learned model 可以作为 imagined rollout generator，帮助训练策略；后半段说明 multi-task structure 可以作为共享与重标注的依据，帮助同一批经验服务于多个任务。两者背后的思想是一致的：**如果环境里存在可复用的结构，就应该想办法把结构转换成数据效率。**

在多任务部分，课程完成了一条非常完整的逻辑链。首先，多任务的真正动机是样本效率与泛化，而不是模型数量管理。其次，任务可以被写成不同 MDP，也可以通过把 task identifier 并入状态，转化为扩展状态空间上的单一 MDP。再次，task conditioning 在实践中可以是 one-hot、语言、示范视频或目标状态。最后，若任务之间共享动力学且奖励可重算，就能使用 hindsight relabeling，把本来只对应单一任务的经验复用到其他任务上。

从研究角度看，这节课留下的最值得继续追问的问题有三个。第一，task representation 应该如何学习，才能真正捕捉任务间共享结构，而不是只记住任务 ID。第二，relabeling 什么时候会引入错误归因，尤其在任务边界不清或奖励函数近似时。第三，随着多任务系统越来越接近 language-conditioned agents，task conditioning 是否会进一步演化为更一般的上下文学习或 preference conditioning。对 CS224R 来说，这正是从经典 RL 走向 modern generalist agent 的关键过渡。